

Rostam Threat Explorer

Implementation handoff — how the diagrams are built, what the data contract is, and the prompts that produce it.

Source of record: `C:\Users\arash\Rostam @ 7f4de02` · written 2026-08-12 · audience: an engineer or agent rebuilding this

1. The short answer

2. Two independent diagram systems

3. System A — Mermaid (LLM-authored)

4. The Phase 2 prompts, verbatim

5. System B — threat-canvas

6. The data contract (threat-graph v1)
7. Core modules

8. Writing a view, end to end

9. Host integration (3 ways)

10. Nine things that burned us

11. Build order for a rebuild

1. The short answer

One runtime diagram dependency: `mermaid ^10.9.0`. That is the whole list.

Everything else — all ten explorer views, the architecture diagram, sankey ribbons, radial layouts, heatmaps, timelines — is **hand-authored SVG and DOM** built with `document.createElementNS` and template strings. No D3, no dagre, no ELK, no cytoscape, no charting library.

```
// frontend/package.json — dependencies, complete
@copilotkit/react-core, @copilotkit/react-ui, @tanstack/react-query,
mermaid, react, react-dom, react-markdown, react-router-dom, remark-gfm
```

WHY THIS MATTERS FOR A REBUILD

The zero-dependency choice is load-bearing, not incidental. The view package is consumed three ways from one compiled source — inside the React SPA, as a run-less import viewer, and as a bare `<script>` embed with no build step. A layout library in the graph path would break the third.

2. Two independent diagram systems

They share no code and solve different problems. Conflating them is the first mistake to avoid.

	SYSTEM A — MERMAID	SYSTEM B — THREAT-CANVAS
Authored by	The LLM, as text, during Phase 2	Code, from structured data
Input	Markdown with <code>```mermaid</code> fences	<code>threat-graph v1</code> JSON
Determinism	None — regenerating changes it	Total — same JSON, same pixels
Failure mode	Syntax error → red error SVG	Missing data → honest empty state
Used for	Data-flow + sequence diagrams in the phase report	The ten interactive explorer views
Count	1 flowchart + ≤3 sequence diagrams	10 registered views

There is a *bridge* between them, and it is the most fragile seam in the system: Phase 10 parses the Phase 2 mermaid back out into graph edges and trust boundaries. Section 10 covers why that hurt.

3. System A — Mermaid (LLM-authored)

Mermaid bakes colors into the SVG string at render time, so it cannot ride on CSS custom properties like the rest of the app. It needs its own palette per theme and a full re-render when the theme changes.

```
// frontend/src/lib/mermaid.ts — the shape that matters
export function initMermaid(theme: Theme = getTheme()) {
  if (initializedFor === theme) return;
  mermaid.initialize({
    startOnLoad: false,
    theme: theme === "dark" ? "dark" : "base",
    themeVariables: THEME_VARIABLES[theme], // full palette, per theme
    securityLevel: "loose",
    flowchart: { useMaxWidth: false, htmlLabels: true },
  });
  initializedFor = theme;
}

export async function renderMermaid(id, source, theme = getTheme()): Promise<string> {
  initMermaid(theme);
  try {
    const { svg } = await mermaid.render(id, source);
    return svg;
  } finally {
    cleanupInjectedDom(id); // <-- see below, this is not optional
  }
}
```

REAL BUG, BUDGET TIME FOR IT

mermaid 10.9.x writes working DOM straight into `document.body` during `render()` — some diagram types do this via their own hardcoded `d3.select("body")`, bypassing any container you pass. On success it cleans up. **On a parse or draw failure it does not**, leaving a red "Syntax error" SVG or an orphaned measurement `<svg>` attached to the page outside React's control. You must remove it yourself, keyed by render id plus mermaid's own wrapper ids (`d{id}`, `i{id}`), plus any id-less `body > svg`.

`useMaxWidth: false` matters too: mermaid otherwise adds a shrink-to-fit style that fights any pan/zoom viewport you wrap it in. Read the true design size from the rendered SVG's `viewBox.baseVal` rather than trusting the element's width.

4. The Phase 2 prompts, verbatim

These are the actual constants from `backend/rostrom/services/phases/phase_prompts.py`. Every rule in them exists because its absence broke something in a real run — they are a scar record, not style preferences.

4.1 Flowchart rules

```
PHASE2_MERMAID_SYNTAX_RULES = """Mermaid syntax rules (violations break rendering):
- Node labels go in square brackets with quotes: `S01["API Gateway"]`.
- Edge labels use pipes and are NEVER quoted: `A -->|steals token| B`. A
  quoted edge label that runs out of room before its closing quote breaks
  the whole diagram; an unquoted one has no closing quote to drop.
- Use `-->` for a one-way data flow and `<-->` for a two-way flow (e.g. a
  request/response pair or a heartbeat/session-check exchanged in both
  directions) – one line, one arrow, one label describing the flow, not two
  separate `-->` lines for the same exchange.
- Every `subgraph` (used for a trust boundary) declares an id immediately
  followed by a bracketed, quoted label: `subgraph TB1["Browser Internet"]`.
  Never emit a `subgraph` with a label and no id – every other component of
  this pipeline that references a trust boundary (node membership, threat
  attribution) does so by that id, not by the label text.
- No parentheses or unquoted special characters inside labels – for edge
  labels that also means no `[ ]` or `{ }`, since mermaid reads those as
  syntax once the label isn't quoted. Commas, slashes, and hyphens are fine.
- Every code fence you open (``) must be closed.
- Edge labels are 6 words or fewer. Spell severity out in full (Low,
  Medium, High) rather than abbreviate it – if the classification is too
  long, shorten the data-type description instead of the severity word."""
```

THE SUBGRAPH-ID RULE IS THE EXPENSIVE ONE

An id-less subgraph "Label" renders perfectly. It is forbidden anyway, because downstream reconciliation references boundaries *by id*. Omitting it is exactly what evicted a real trust boundary from a synthesized graph. A rule can be about your pipeline, not about the renderer — say which, in the prompt's own comment, or someone will "simplify" it later.

4.2 Sequence-diagram rules — deliberately different

Sequence-diagram syntax is *not* flowchart syntax. Message text runs unquoted to end-of-line, so the quoting and bracket rules above actively harm here. This was established by driving the real mermaid parser, not by reading docs.

```
PHASE2_SEQUENCE_SYNTAX_RULES = """Mermaid sequence-diagram syntax rules (violations break rendering) – these
are NOT the same rules as the flowchart rules above:
- Declare each participant once, reusing the exact S0X/D0X id from the
  component inventory as the participant id: `participant S02 as Auth Service`.
  Never introduce a participant that isn't an existing S0X/D0X id.
- Message text follows a colon and runs unquoted to the end of the line –
  do not wrap it in quotes: `S01->>S02: submit credentials`. Parentheses,
  brackets, braces, colons, and quote characters are all safe unquoted in
  that text.
- Never put a semicolon (;) in message text – mermaid treats it as a
  statement separator and it breaks the parse even mid-message. Use "and"
  or a comma instead.
- Use `->>` for a request/call and `-->>` for its response. Use
  `Note over S0X,S0Y: ...` for context that isn't a message between two
  participants. `activate`/`deactivate` are optional, for showing a
  participant is busy handling a prior request.
- Every code fence you open (``) must be closed.
- Keep each diagram to the steps where order matters – 4 to 8 messages is
  enough; omit steps that don't affect correctness or security."""
```

4.3 The Phase 2 system prompt

```
PHASE2_SYSTEM = f"""You are running Phase 2: Data-flow analysis.

Task: Produce a Mermaid `flowchart LR` diagram showing every data flow between
the components identified in Phase 1, labeled with the data classification
crossing each edge. Annotate trust boundaries. Then, separately, diagram the
ordering of the handful of flows where sequence is a security property.

Output, in this order:

## Data Flow
A single mermaid block with the `flowchart LR` diagram described above.

## Trust Boundaries
A table listing each boundary and which components sit on each side.

## Key Flow Sequences
At most 3 mermaid `sequenceDiagram` blocks, one per flow. A flowchart shows
what connects to what; it cannot show order, so reserve this section for
flows whose correctness depends on doing steps in the right order –
authentication, token issuance/validation, privilege or tenant-boundary
crossings, secret retrieval, or anything else where the security property
IS the order (validate-before-use, verify-signature-before-trust,
check-expiry-before-accept). Do not diagram routine data pass-through with
no ordering constraint – the Data Flow diagram already covers that, and an
exhaustive catalogue of request paths buries the flows that actually matter.
Prefer 1-2 tightly chosen sequences over 3 loosely chosen ones; emit zero if
no flow in this system has order-dependent, security-relevant behavior.
Every participant must be an S0X or D0X id already assigned in Phase 1.

{PHASE2_MERMAID_SYNTAX_RULES}

{PHASE2_SEQUENCE_SYNTAX_RULES}
"""
```

Two design decisions worth copying: the **cap** (at most 3) and the **selection criterion** ("the security property IS the order"). Without both, the model emits a sequence diagram per request path and buries the ones that matter.

5. System B — threat-canvas

A framework-agnostic, dependency-free TypeScript package. Ten views, plain DOM/SVG.

```
frontend/src/threat-canvas/
├─ index.ts           public API + side-effect view registration
├─ registry.ts        registerView / renderThreatView / getRegisteredViews
├─ types.ts           threat-graph v1 TypeScript mirror of the JSON schema
├─ events.ts          4 outbound CustomEvents – the only host coupling
├─ escape.ts          esc() – mandatory on every graph-derived string
├─ severity.ts        severity → CSS var + rank
├─ format.ts          evidence / MITRE id formatting
├─ relevance.ts       "is this node worth showing by default"
├─ element.ts         <rosteam-threat-view> custom element
├─ theme.css          --tc-* design tokens (light + dark)
├─ interaction/
│   └─ panZoom.ts     generic viewport/content pan + zoom controller
├─ views/             10 × (view.ts + view.css)
├─ fixtures/          canonical + adversarial graphs
└─ __tests__          registry-driven acceptance suite
```

The ten views

VIEW	LINES	ANSWERS
threat-lens	1042	What is the threat and where does it live?
attack-path	946	How does an attacker get from entry to crown jewel?
sensitivity-sankey	938	Which data flows are fattest and most sensitive?
crown-jewel	826	How many hops is an attacker from what matters?
trust-zones	721	What crosses which trust boundary?
triage-minimap	658	Where on the map is this threat?
stride-heatmap	474	Which STRIDE categories cluster where?
mitre-lanes	448	Which ATT&CK tactics are covered?
evolution-timeline	377	How did findings change across phases?
layered-tilt	319	How do C4 levels stack?

6. The data contract (threat-graph v1)

Everything downstream of Phase 10 speaks this. The TS types mirror `schema/threat-graph/threat-graph.v1.schema.json` 1:1.

```

export interface ThreatGraph {
  schemaVersion: "1.0";
  meta:      GraphMeta;           // { generator, appName, runId?, createdAt }
  levels:    Record<string, LevelDef>;
  nodes:     GraphNode[];
  edges:     GraphEdge[];
  boundaries: Boundary[];
  threats:   Threat[];
  actors:    Actor[];
  evolution?: EvolutionEvent[];
  reconciliation?: Reconciliation;
}

export interface GraphNode {
  id: string;           // "S01" service | "D01" datastore
  label: string;
  level: "context" | "container" | "component";
  service: string;      // "api" | "database" | "cdn" | ...
  severity: QualitativeSeverity;
  threats: string[];    // threat ids touching this node
  boundary: string;     // boundary id, or "TB-UNASSIGNED"
  evidence: EvidenceRef; // { file, startLine, endLine }
  parent?: string;
  crownJewel?: boolean;
}

export interface GraphEdge {
  from: string; to: string; label: string;
  sensitivity: "high" | "med" | "low";
  crossesBoundary: boolean;
  carriesSecret?: boolean;
}

export interface Threat {
  id: string; fingerprint: string; title: string;
  actor: string;           // actor id
  stride: string;
  severity: QualitativeSeverity;
  evidence: EvidenceRef;
  path: string[];         // ordered ids: actor → ... → target
  verdict: "confirmed" | "challenged" | "suggested_remove" | "suggested";
  mitre?: MitreEntry[]; cwe?: string[]; capec?: string[];
  controls?: Control[];
  likelihood?: number; impact?: number; riskScore?: number;
}

```

THE TWO FIELDS EVERYTHING HINGES ON

`threat.path` is what the attack-path and sankey views trace. A graph without paths renders nodes but no routes, and looks broken rather than empty.

`node.threats` drives the default relevance filter (section 7). A graph where nobody populates it will show every node at once, which on real data is unreadable.

7. Core modules

7.1 Registry — the whole plugin surface

```
export interface ViewHandle {
  el: HTMLElement;
  update(graph: ThreatGraph): void;    // re-render in place
  destroy(): void;                     // tear down listeners/timers, clear DOM
}

export type ViewFactory =
  (el: HTMLElement, graph: ThreatGraph, options?: RenderOptions) => ViewHandle;

export function registerView(name: string, factory: ViewFactory): void;
export function getRegisteredViews(): string[];

export function renderThreatView(el, view, graph, options?): ViewHandle {
  const factory = registry.get(view);
  if (!factory) throw new Error(
    `threat-canvas: unknown view "${view}" (registered: ${getRegisteredViews().join(", ")}`);
  el.innerHTML = "";
  return factory(el, graph, options);
}
```

Views self-register as an import side effect, so `index.ts` just imports them. Adding a view is one file plus one import line — no switch statement to update.

7.2 Events — the only coupling to a host

threat-canvas never imports host code. It talks outward through four DOM CustomEvents, dispatched on the mount element with `bubbles: true, composed: true`.

```
"threat-select" { threatId }
"node-select"   { nodeId }
"evidence-open" { file, startLine, endLine }
"drill"         { fromLevel, toLevel, viaId }
```

7.3 `esc()` — a hard security requirement

NON-NEGOTIABLE

The package renders **arbitrary untrusted JSON** — pasted documents, possibly produced by another AI. Every string originating from graph data must pass through `esc()` before landing in an HTML template, in text position or inside a double-quoted attribute. There is an adversarial fixture (`fixtures/hostile-payload.threatgraph.json`) where every string field carries an XSS attempt; it must render inert. Keep that fixture and keep it in CI.

7.4 Design tokens

All colour lives in `--tc-*` custom properties so an embedder can retheme without touching view code. Severity never appears as a hex in a view.

```

--tc-bg / --tc-panel / --tc-raised / --tc-deep          surfaces
--tc-hair / --tc-card-fill / --tc-card-border          structure
--tc-cyan / --tc-violet / --tc-attn                    accents
--tc-sev-veryhigh #ff7d6b  --tc-sev-high   #f5a08c
--tc-sev-medium   #ecc55f  --tc-sev-low    #6fbf8f
--tc-sev-verylow  #4fae9d
--tc-text #e2e6f5 / --tc-text-2 #8b93b8 / --tc-text-3 #565e84
--tc-*-rgb variants for rgb(var(--x) / alpha) compositing

```

7.5 relevance.ts — the filter that makes real graphs readable

```

export function isThreatRelevant(node: Pick<GraphNode, "threats">): boolean {
  return (node.threats?.length ?? 0) > 0;
}

/** "16 of 83 components shown — 67 with no threats hidden" */
export function relevanceStatus(shown: number, total: number, noun = "components"): string;

/** Same button copy in every view, so the toggle reads as one system. */
export function relevanceToggleLabel(showingAll: boolean, total: number): string {
  return showingAll ? "Focus on threats" : `Show all ${total} components`;
}

```

A real run's graph is dominated by structural plumbing no threat ever reaches — one live run has 83 nodes of which 16 carry a threat. Drawing all of it by default buries what the views exist to surface. Default to the relevant subset, expose one identically-worded toggle, and **always state the hidden count** — never filter silently.

7.6 panZoom.ts — one controller, every pannable surface

```

export interface PanZoomHandle {
  getState(): { scale: number; tx: number; ty: number };
  setScale(scale: number, anchorClientX?: number, anchorClientY?: number): void;
  fitToContent(w: number, h: number, padding?: number, maxFitScale?: number): void;
  reset(): void;
  destroy(): void;
}

createPanZoom(viewport: HTMLElement, content: HTMLElement, options?): PanZoomHandle;

```

Pattern: `viewport` is `overflow:hidden`, `content` is absolutely positioned with `transform-origin: 0 0` and gets its transform driven. Elements that must not start a pan (buttons, clickable nodes) opt out with class `tc-no-pan`.

`maxFitScale` defaults to 1 — `fitToContent` will shrink to fit but not magnify, so a small graph renders at native size rather than being blown up.

8. Writing a view, end to end

Every view follows the same seven-part shape. This is the actual skeleton, abridged from `views/layeredTilt.ts` .

```

// 1. Imports – types, registry, and the shared helpers. Never host code.
import type { GraphNode, ThreatGraph } from "../types";
import { registerView, type ViewHandle } from "../registry";
import { esc } from "../escape";
import { emit } from "../events";
import { severityVar, severityRank } from "../severity";
import { formatEvidence } from "../format";
import "../myView.css";

const VIEW_NAME = "my-view";
const ROOT_CLASS = "tc-view-my-view";

// 2. Static skeleton – no graph data in here, so no escaping needed yet.
function buildSkeleton(): string {
  return `
    <div class="${ROOT_CLASS}">
      <div class="mv-caption"><b>My View</b> – what question it answers</div>
      <div class="mv-stage" data-role="stage"></div>
      <div class="mv-side" data-role="side"></div>
    </div>`;
}

export function mountMyView(hostEl: HTMLElement, initialGraph: ThreatGraph): ViewHandle {
  let cleanupKeydown: (() => void) | null = null;

  function mount(graph: ThreatGraph): void {
    hostEl.classList.add("tc-root");
    hostEl.innerHTML = buildSkeleton();

    const q = <T extends Element>(role: string): T => {
      const found = hostEl.querySelector<T>(`[data-role="${role}"]`);
      if (!found) throw new Error(`threat-canvas/${VIEW_NAME}: missing [data-role="${role}"]`);
      return found;
    };
    const stage = q<HTMLDivElement>("stage");

    // 3. Honest empty state BEFORE any layout maths.
    if (graph.nodes.length === 0) {
      stage.innerHTML = `<div class="mv-empty">No components in this graph yet.</div>`;
      return;
    }

    // 4. Layout computed FROM the graph. Never a hardcoded coordinate table.
    const shown = graph.nodes.filter(isThreatRelevant);
    const geo = computeLayout(shown, graph.edges);

    // 5. Render. Every graph-derived string through esc().
    shown.forEach((n) => {
      const g = svgEl("g", { class: "mv-node tc-no-pan", tabindex: 0, role: "button",
        "aria-label": `${n.id} ${n.label}, severity ${n.severity}` });
      g.appendChild(svgEl("rect", { x: geo[n.id].x, y: geo[n.id].y,
        width: 152, height: geo[n.id].h,
        stroke: severityVar(n.severity) }));
      const t = svgEl("text", { x: geo[n.id].x + 11, y: geo[n.id].y + 16 });
      t.textContent = n.label; // textContent is safe; innerHTML needs esc()
      g.appendChild(t);

      // 6. Talk to the host only through events.
      g.addEventListener("click", () => emit(hostEl, "node-select", { nodeId: n.id }));
      stage.appendChild(g);
    });

    // ESC clears selection – but only claims the key when it cleared something,
    // so an enclosing fullscreen overlay still gets ESC when nothing is selected.

```

```

const onKeydown = (e: KeyboardEvent) => {
  if (e.key !== "Escape" || selection === null) return;
  e.preventDefault();
  clearSelection();
};
document.addEventListener("keydown", onKeydown);
cleanupKeydown = () => document.removeEventListener("keydown", onKeydown);
}

mount(initialGraph);

// 7. Lifecycle – update re-mounts, destroy leaves nothing behind.
return {
  el: hostEl,
  update(graph) { cleanupKeydown?(); mount(graph); },
  destroy() { cleanupKeydown?(); hostEl.innerHTML = ""; },
};
}

registerView(VIEW_NAME, (el, graph) => mountMyView(el, graph));

```

The SVG helper every view defines

```

const SVGNS = "http://www.w3.org/2000/svg";
function svgEl<K extends keyof SVGElementTagNameMap>(
  tag: K, attrs: Record<string, string | number>
): SVGElementTagNameMap[K] {
  const el = document.createElementNS(SVGNS, tag);
  for (const k in attrs) el.setAttribute(k, String(attrs[k]));
  return el as SVGElementTagNameMap[K];
}

```

9. Host integration — three ways, one source

9.1 React (the SPA)

```

export default function ThreatViewHost({ view, graph, onNodeSelect, onEvidenceOpen }) {
  const hostRef = useRef<HTMLDivElement>(null);
  const callbacksRef = useRef({ onNodeSelect, onEvidenceOpen });
  callbacksRef.current = { onNodeSelect, onEvidenceOpen }; // no stale closures

  useEffect(() => {
    const el = hostRef.current;
    if (!el) return;
    const handle = renderThreatView(el, view, graph);

    const listeners: Array<[string, (e: Event) => void]> = [
      ["node-select", (e) => callbacksRef.current.onNodeSelect?.((e as CustomEvent).detail)],
      ["evidence-open", (e) => callbacksRef.current.onEvidenceOpen?.((e as CustomEvent).detail)],
    ];
    listeners.forEach(([t, fn]) => el.addEventListener(t, fn));

    return () => {
      listeners.forEach(([t, fn]) => el.removeEventListener(t, fn));
      handle.destroy();
    };
  }, [view, graph]);

  return <div ref={hostRef} className="h-full w-full" />;
}

```

9.2 Custom element (no framework)

```
<rostatm-threat-view view="threat-lens"></rostatm-threat-view>
<script type="module">
  import "threat-canvas";           // self-registers the element
  document.querySelector("rostatm-threat-view").graph = myGraph; // property, not attribute
</script>
```

`view` is an observed attribute so it works declaratively; `graph` is a property because JSON is too large for an attribute.

9.3 Direct call

```
const handle = renderThreatView(hostElement, "threat-lens", graph);
handle.update(newGraph);
handle.destroy();
```

10. Nine things that burned us

The most valuable section. Each of these shipped, looked fine, and was wrong.

1 — Hardcoded coordinate tables that only work on the demo fixture

Two views shipped with hand-placed `POS` tables (19 ids) ported from the original mockup. On any other graph most nodes silently never appeared — 19 of 83 on one run — and the "zone boxes" were decorative rather than the graph's real boundaries. It stayed invisible because the only fixture anyone tested with was the mockup the table was copied from, where broken and correct produce *identical* output. **Compute layout from the graph, always. Assert coverage as a property (`expect(drawn).toBe(graph.nodes.length)`), never as a pinned number.**

2 — A parser that only worked because of a bug in its input

Phase 10 parses Phase 2's mermaid back into edges. One of its regexes existed to absorb an *unterminated-quote defect* in the LLM output. Fixing the prompt so the output was correct made it simultaneously unparseable — the run went to **0 edges and 1 boundary**, and four views rendered empty. If a downstream parser consumes LLM output, changing the prompt is a breaking change to the parser. Version them together.

3 — Nodes with no edges destroying a sankey

62 of 83 nodes had no ribbon at all. With nothing pointing at them they all landed in layer 0, and that one column stacked to **2900px inside a 640px frame**. Lay out only nodes that carry flow, and state the omitted count.

4 — Fit-to-content on an extreme aspect ratio

22 nodes across 13 stages is naturally an 8:1 band. Fitting that into a 16:9 frame fits on width and wastes 80% of the height. Stretch the layout toward the frame's proportions — scaling node heights *and* ribbon widths by the same factor, so every encoded proportion is preserved.

5 — Column pitch narrower than the node

When tightening horizontal pitch for deep graphs, floor it at `NODE_W + gap`. A naive floor overlaps every column.

6 — ESC belonging to two owners

Three views bind `ESC` on `document` to clear selection; a fullscreen overlay also exits on `ESC`. One press did both. Fix: views call `preventDefault()` *only when they actually cleared something*, and the outer handler listens on `window` — events bubble `document` → `window`, so it is always last regardless of bind order.

DO NOT SOLVE THIS WITH A MICROTASK

Deferring the outer check to `queueMicrotask` passes in jsdom and fails in a real browser. For a synthetic `dispatchEvent` all listeners run inside one script frame, so microtasks drain after all of them. For a **trusted keypress the browser takes a microtask checkpoint after each listener**, so the deferred check runs before the inner handler is ever called. Use the structural guarantee (`document` → `window`), not a timing one — and drive the real bubble path in the test.

7 — Citation paths off by one directory

Models cite relative to the repo root (`app/main.py`) while the upload root holds the zip's wrapper directory (`MyApp/app/main.py`) — and they are not consistent: one run resolved 40/40, another 0/33, a third emitted both forms in the same document. **119 of 165 citations were unresolvable**. Fix in the resolver (retry under each top-

level directory, same containment check, report which path actually opened), not only in the prompt — a resolver fix is retroactive to runs already on disk.

8 — Reading a build's exit code through a pipe

`docker compose up --build 2>&1 | tail -8` reports *tail's* exit code. A failed build was reported as a successful deploy. Capture it: `cmd > log 2>&1; echo $? > exit`. And verify inside the running container, not from the build log.

9 — Empty states that lie

A findings tab read a variable only populated while a *different* tab was open, so a completed run rendered "No findings yet (Phase 7 not complete)". Distinguish *not run yet* from *ran and produced nothing* from *ran but could not be parsed*. They are three different bugs and one message hides all three.

11. Build order for a rebuild

1. **Schema first.** Write the JSON schema and the TS mirror before any view. Everything negotiates through it.
2. **Registry + one view.** Prove `renderThreatView` → `ViewHandle` → `destroy` with the simplest view.
3. **Two fixtures, immediately.** A canonical graph and an adversarial one where every string is an XSS attempt. Also keep a *degraded* fixture (few edges, one boundary) — most layout bugs only appear there.
4. **esc() everywhere, from the first line.** Retrofitting escaping across ten views is far worse than starting with it.
5. **Tokens before colour.** No hex in a view file, ever.
6. **Remaining views.** Each: honest empty state → graph-derived layout → events → lifecycle.
7. **A registry-driven acceptance suite** that fails if a view is registered without acceptance config — so view eleven cannot skip the checks.
8. **panZoom + fullscreen last**, once you know which surfaces exceed their frame.

IF YOU TAKE ONE THING

Test every view against a graph that is *not* the one the design was drawn from. Three separate classes of bug in this system — hardcoded coordinates, the sankey stacking, the empty explorer — were all invisible on the demo fixture and obvious on the first real run.